

uC/OS-II

目录

uC/OS-II

目录

- 一、系统概述
- 二、任务和任务状态
- 三、优先级和调度
- 四、任务栈和任务控制块
- 五、系统节拍和时间管理
- 六、中断和临界区
- 七、任务同步和通信
 - 1. 信号量
 - 2. 互斥量
 - 3. 邮箱
 - 4. 消息队列
 - 5. 事件标志组
- 八、常用函数
 - 1. 内核启动与中断管理
 - 2. 任务创建
 - 3. 任务控制
 - 4. 时间管理
 - 5. 信号量
 - 6. 互斥量
 - 7. 邮箱
 - 8. 消息队列
 - 9. 事件标志
- 九、移植
- 十、常见问题

一、系统概述

uC/OS-II是一个面向嵌入式系统的实时操作系统内核。它的主要作用是把复杂程序拆分成多个任务，并通过优先级、就绪表、系统节拍和同步对象管理任务运行顺序。

实时操作系统强调“在规定时间内响应事件”。这里的实时不等于运行速度一定最快，而是任务响应时间具有可分析性。对于周期采集、通信处理、按键扫描、状态刷新等场景，实时操作系统可以使程序结构更清楚，也便于给不同工作分配不同优先级。

裸机程序通常把所有逻辑放在一个主循环中：

```
while (1)
{
    TaskA_Process();
    TaskB_Process();
    TaskC_Process();
}
```

这种方式简单，但所有函数都必须快速返回。某个函数阻塞时，后续函数也会被拖住。uC/OS-II把这些工作拆成任务，由调度器决定当前运行哪个任务。单核CPU并不会真正同时运行多个任务，而是在任务切换时保存当前任务现场，再恢复另一个任务现场，使多个任务交替执行。

注意事项：

- 实时操作系统解决的是任务组织、调度和同步问题，不会自动解决业务逻辑错误；
- 使用操作系统后，延时、共享变量、中断处理和任务优先级都需要重新设计；
- 任务数量越多，结构不一定越好，应按职责划分，避免把简单流程拆得过碎；
- 实时性来自清晰的优先级和有限的阻塞时间，不是来自无限提高CPU频率。

二、任务和任务状态

任务是uC/OS-II调度的基本单位。每个任务通常由一个不会返回的函数表示，并拥有独立的任务栈和任务控制块。

```
static void AppTask(void *p_arg)
{
    (void)p_arg;

    for (;;)
    {
        /* 周期处理或等待事件 */
        OSTimeDly(1u);
    }
}
```

任务入口函数通常不能返回。如果任务函数返回，任务栈和任务控制块状态可能无法按预期维护。

状态	含义	常见进入条件
运行态	当前正在占用CPU	调度器选择该任务
就绪态	可以运行，等待CPU	优先级低于当前运行任务
延时态	暂时不能运行	调用 OSTimeDly() 等时间函数
等待态	等待事件或资源	调用信号量、邮箱、队列等 Pend 函数
挂起态	被人为暂停	调用 OSTaskSuspend()
删除态	不再参与调度	调用 OSTaskDel()

同一时刻只有一个任务处于运行态，但可以有多个任务处于就绪态、延时态或等待态。

注意事项：

- 任务函数中不能写没有阻塞点的死循环，否则低优先级任务可能长期得不到CPU；
- 延时态和等待态都会让出CPU，但含义不同：延时是等时间，等待是等事件；
- 任务状态变化由内核维护，不应由应用程序直接修改任务控制块；
- 任务入口参数是 void *，传递结构体指针时要保证其生命周期有效。

三、优先级和调度

uC/OS-II采用基于优先级的抢占式调度。优先级数值越小，任务优先级越高。高优先级任务一旦变为就绪态，就可以抢占低优先级任务。

uC/OS-II通常要求每个任务使用唯一优先级。优先级既是任务紧急程度的表达，也是调度器查找就绪任务的重要依据。

高优先级任务就绪

-> 调度器比较当前任务与最高就绪优先级

-> 需要切换时保存当前任务现场

-> 恢复高优先级任务现场

-> 高优先级任务继续执行

优先级设计应遵循以下原则：

- 对响应时间要求高、处理时间短的任务优先级较高；
- 周期性状态刷新、显示更新等任务优先级较低；
- 低优先级任务不能长期持有高优先级任务需要的资源；
- 高优先级任务完成一次处理后应进入延时或等待状态。

注意事项：

- 优先级高不代表任务应一直运行；
- 优先级设置不当会造成任务饥饿，即低优先级任务长期无法执行；
- 中断优先级和任务优先级不是同一套概念，不能混为一谈；
- 抢占式调度提高响应能力，同时也引入共享数据保护问题。

四、任务栈和任务控制块

每个任务都有独立栈。任务栈用于保存局部变量、函数调用返回地址、任务切换时的寄存器现场，以及部分CPU异常进入时自动压入的现场。

栈使用来源	说明
局部变量	局部数组和大型结构体会明显增加栈占用
函数调用	调用层级越深，需要保存的返回现场越多
中断现场	异常进入时CPU会自动保存部分寄存器
浮点现场	使用硬件浮点时可能增加现场保存内容

任务控制块通常保存任务状态、优先级、栈指针、延时计数、事件等待信息等。调度器不是直接“记住函数执行到哪里”，而是通过任务栈和任务控制块恢复任务上下文。

注意事项：

- 栈太小会导致越界、HardFault或随机数据破坏；
- 栈过大会浪费RAM，嵌入式系统RAM通常有限；
- 栈大小单位可能是 `OS_STK` 元素数量，不一定等于字节数；
- 递归、大型局部数组和复杂格式化输出都容易增加栈风险。

五、系统节拍和时间管理

系统节拍是内核的时间基准，通常由定时器周期中断产生。每个节拍到来时，内核更新系统时间，并减少延时任务的等待计数。计数到期后，任务重新进入就绪态。

常用时间函数包括：

函数	作用
<code>OSTimeDly()</code>	按节拍数延时当前任务
<code>OSTimeDlyHMSM()</code>	按时、分、秒、毫秒形式延时
<code>OSTimeGet()</code>	获取当前系统节拍计数
<code>OSTimeSet()</code>	设置系统节拍计数

`OSTimeDly()` 不是忙等待。任务调用后会进入延时态，CPU可以运行其他就绪任务。忙等待则是当前任务一直占用CPU等待时间过去。

注意事项：

- 节拍频率越高，时间分辨率越细，但中断开销也越大；
- 毫秒级延时能否准确表达，取决于系统节拍频率；
- 系统节拍计数可能溢出，时间差比较应考虑无符号回绕；
- 不能在中断服务函数中调用会阻塞的时间延时函数。
-

六、中断和临界区

中断用于响应外部事件或定时事件。uC/OS-II中，若中断需要使用内核服务，通常应在中断入口和出口处配合调用中断管理接口：

```
OSIntEnter();
/* 中断处理 */
OSIntExit();
```

`OSIntEnter()` 用于告诉内核当前进入中断嵌套状态，`OSIntExit()` 用于在中断退出时判断是否需要进

行任务切换。中断服务函数应尽量短，只完成保存数据、清除标志、发送通知等必要工作。

临界区用于保护共享数据：

```
OS_CPU_SR cpu_sr;

OS_ENTER_CRITICAL();
/* 访问共享变量 */
OS_EXIT_CRITICAL();
```

临界区通常会暂时屏蔽中断或调度，因此范围必须短小。不能在临界区中执行延时、等待信号量、长循环或耗时输出。

注意事项：

- 中断中不能调用会阻塞的 Pend 类函数；
- 中断可以发送事件通知，但应避免执行复杂业务处理；
- 进入临界区后必须保证所有路径都能退出；
- 临界区保护的是短时间共享访问，不适合代替任务同步机制。

七、任务同步和通信

任务同步用于解决“什么时候继续运行”的问题，任务通信用于解决“数据怎样传递”的问题。uC/OS-II 常见同步和通信对象包括信号量、互斥量、邮箱、消息队列和事件标志组。

对象	适合场景
信号量	事件通知、资源计数
互斥量	独占访问共享资源
邮箱	传递一个消息指针
消息队列	缓存多个消息指针
事件标志组	等待一个或多个条件位

1.信号量

信号量可以表示事件是否发生，也可以表示某类资源的可用数量。

```
OS_EVENT *sem;

创建信号量，参数是信号量资源初始值
sem = OSemCreate(0u);
if (sem == NULL)
{
    Error_Handler();    创建失败，系统不可用
}

uint8_t err;
OSemPend(sem, timeout, &err);
if (err == OS_ERR_NONE)
{
    信号量被释放，说明可以开始工作
}
else if (err == OS_ERR_TIMEOUT)
{
    超时说明对方没发信号,进行超时处理
}
else
{
    Error_Handler();    其他错误，进行错误处理
}

OSemPost(sem);
```

创建信号量时，初值为 0 常用于事件通知，表示事件尚未发生；初值大于 0 常用于计数资源。等待任务调用 `OSSemPend()`，若计数大于 0，则计数减 1 后继续运行；若计数为 0，则任务进入等待态。事件发生后调用 `OSSemPost()`，唤醒等待任务或增加计数。

不使用信号量时，任务往往只能不断轮询标志。轮询周期太短会浪费 CPU，周期太长会增加响应延迟。信号量使任务在没有事件时进入等待态，事件到来后再被唤醒。

注意事项：

- 信号量只表示事件或资源数量，不保存具体数据内容；
- 数据本身应存放在缓冲区、结构体或队列中；
- `OSSemPend()` 不能在中断中调用，因为中断不能阻塞；
- 等待信号量应检查返回错误码和超时结果。

2. 互斥量

互斥量用于保护同一时间只能被一个任务访问的共享资源。与普通信号量相比，互斥量强调所有权，通常由获得互斥量的任务释放。

```
定义互斥量句柄
OS_EVENT *mutex;

/* 2. 创建互斥量（通常在 main 或任务初始化时调用） */
INT8U err;
mutex = OSMutexCreate(1, &err); // 1 是优先级继承优先级（需高于所有使用该互斥量的任务）
if (err != OS_ERR_NONE)
{
    // 创建失败，系统无法运行，触发错误处理
    Error_Handler();
}
mutex = OSMutexCreate(prio, &err);
OSMutexPend(mutex, timeout, &err);
/* 访问共享资源 */
OSMutexPost(mutex);
```

互斥量常用于保护共享数据结构、打印输出、外设访问接口等。多个优先级不同的任务共享资源时，互斥量还能配合优先级继承或优先级上限机制降低优先级反转风险。

3. 邮箱

邮箱用于传递一个指针。发送方调用 `OSMboxPost()`，接收方调用 `OSMboxPend()`。邮箱适合只传递一条消息或只关心最新指针的场景。

注意事项：

- 邮箱传递的是指针，不会自动复制指针指向的数据；
- 指针指向的数据必须在接收方使用期间保持有效；
- 需要缓存多条消息时，应使用消息队列。

4. 消息队列

消息队列可以保存多个消息指针，适合生产速度和消费速度不完全一致的场景。接收任务可以在队列为空时等待，发送任务可以在事件发生时把消息放入队列。

注意事项：

- 队列容量有限，满队列时必须明确处理策略；
- 队列通常保存指针，不负责管理指针所指数据的生命周期；
- 队列适合传递事件数据，不适合保护共享资源的互斥访问。

5. 事件标志组

事件标志组使用多个bit表示多个条件，可以等待任意一个条件满足，也可以等待多个条件同时满足。它适合组合条件同步，例如等待“数据准备好”和“通信允许”两个条件。

注意事项：

- 等待条件应明确是“任一满足”还是“全部满足”；
- 事件满足后是否自动清除需要明确；
- 标志位长期不清除可能导致任务重复认为事件已经发生。

八、常用函数

以下接口以uC/OS-II V2.93为准。INT8U、INT16U 和 INT32U 分别表示8位、16位和32位无符号整数。事件对象通常以 OS_EVENT * 表示，事件标志组使用 OS_FLAG_GRP *。这些指针由创建函数返回，应用程序只保存和传递，不直接修改其内部成员。

perr 用于输出错误码，调用后必须检查 *perr。OS_ERR_NONE 表示成功。timeout 的单位是系统节拍，值为 0 表示永久等待，不表示立即返回。需要立即尝试而不阻塞时，应使用相应的 Accept 函数。

1. 内核启动与中断管理

```
void OSInit(void);
void OSStart(void);
void OSIntEnter(void);
void OSIntExit(void);
```

函数	参数与返回值	说明
OSInit()	无参数、无返回值	初始化内核全局变量、就绪表、任务控制块池、事件控制块池，并创建空闲任务。必须在创建其他内核对象之前调用，且只能初始化一次。
OSStart()	无参数、无返回值	启动调度并运行最高优先级就绪任务。正常情况下不返回；返回通常表示尚无可运行任务或移植层启动代码异常。
OSIntEnter()	无参数、无返回值	中断入口调用，使中断嵌套计数加1。它不是开中断函数，也不负责清除外设中断标志。
OSIntExit()	无参数、无返回值	中断出口调用，使嵌套计数减1；最后一层中断退出时，若有更高优先级任务就绪，则进行调度。

`OSIntEnter()` 与 `OSIntExit()` 必须成对使用。中断服务函数中只能调用明确允许在中断环境使用的接口，不能调用任何可能阻塞当前执行流的 `Pend` 函数。

```
OS_CPU_SR cpu_sr;

OS_ENTER_CRITICAL();
/* 短时间访问共享数据 */
OS_EXIT_CRITICAL();
```

`OS_ENTER_CRITICAL()` 和 `OS_EXIT_CRITICAL()` 是移植层宏，没有普通函数意义上的参数和返回值。部分移植方式要求当前函数定义 `OS_CPU_SR cpu_sr`，用于保存进入临界区前的中断状态。退出时恢复原状态，而不是无条件开中断。临界区内不能延时、等待事件、执行长循环或进行耗时输出。

2. 任务创建

```
INT8U OSTaskCreate(void (*task)(void *p_arg),
                  void *p_arg,
                  OS_STK *ptos,
                  INT8U prio);

INT8U OSTaskCreateExt(void (*task)(void *p_arg),
                     void *p_arg,
                     OS_STK *ptos,
                     INT8U prio,
                     INT16U id,
                     OS_STK *pbos,
                     INT32U stk_size,
                     void *pext,
                     INT16U opt);
```

参数	含义
task	任务入口函数地址，函数形式必须为 <code>void Task(void *p_arg)</code> 。任务通常包含无限循环，不能执行到函数末尾。
p_arg	首次进入任务时传给任务函数的参数。无参数时传 <code>NULL</code> ；传指针时，其指向对象必须在任务使用期间持续有效，不能指向已经退出的函数的局部变量。
ptos	任务初始栈顶。栈向低地址增长时通常传 <code>&stack[STACK_SIZE - 1u]</code> ；栈向高地址增长时传 <code>&stack[0]</code> 。它必须指向栈数组内一个有效元素。
prio	任务优先级，数值越小优先级越高。每个任务必须使用唯一优先级，不能占用空闲任务、统计任务或互斥量优先级上限所保留的优先级。
id	用户任务编号，范围为 0 至 65535。它用于识别或统计，不参与调度，可设为与任务相关的固定编号。通常和唯一的任务优先级保持一致
pbos	栈底地址。栈向低地址增长时通常为 <code>&stack[0]</code> ，栈向高地址增长时为数组最高地址。
stk_size	栈中 <code>OS_STK</code> 元素的数量，不一定是字节数。

参数	含义
pext	用户扩展数据指针，内核把它保存在任务控制块中。没有扩展数据时传 NULL，其生命周期要求与 p_arg 相同。
opt	任务选项。OS_TASK_OPT_NONE 表示无附加选项；OS_TASK_OPT_STK_CHK 允许栈检查；OS_TASK_OPT_STK_CLR 在创建时清零栈；OS_TASK_OPT_SAVE_FP 要求切换时保存浮点寄存器。多个选项用按位或组合。

返回值为错误码。OS_ERR_NONE 表示创建成功；OS_ERR_PRIO_EXIST 表示优先级已占用；OS_ERR_PRIO_INVALID 表示优先级越界；OS_ERR_TASK_CREATE_ISR 表示在中断中创建任务；OS_ERR_TASK_NO_MORE_TCB 表示任务控制块已用完。

OSTaskCreateExt() 适合需要栈检查或任务扩展信息的场合。任务创建成功后，如果调度器已经运行且新任务优先级更高，新任务可能在创建函数返回前抢占当前任务。创建函数只建立任务，不负责任务之间的同步。

栈检查 是操作系统提供的一种安全机制，用来检测**任务栈的使用情况**，比如栈还剩多少空间，或者有没有溢出。

在 UCOS 里，栈检查通常通过 OSTaskStkChk() 函数实现。它会遍历整个任务栈，统计已用的栈空间和剩余的栈空间，返回给调用者。

3. 任务控制

```

INT8U OSTaskSuspend(INT8U prio);
INT8U OSTaskResume(INT8U prio);
INT8U OSTaskDel(INT8U prio);
INT8U OSTaskChangePrio(INT8U oldprio, INT8U newprio);
INT8U OSTaskStkChk(INT8U prio, OS_STK_DATA *p_stk_data);

```

函数	参数	返回值与作用
OSTaskSuspend()	prio 为目标任务优先级， OS_PRIO_SELF 表示当前任务	成功返回 OS_ERR_NONE 并使任务进入挂起状态。空闲任务不能挂起；目标不存在、优先级非法时返回相应错误。挂起自身后，函数要到任务被恢复时才继续向下执行。
OSTaskResume()	prio 为待恢复任务优先级，不接受 OS_PRIO_SELF	只解除一次挂起状态。成功返回 OS_ERR_NONE；任务未挂起返回 OS_ERR_TASK_NOT_SUSPENDED；目标不存在返回 OS_ERR_TASK_NOT_EXIST。若任务还在延时或等待事件，解除挂起后仍不会立即就绪。
OSTaskDel()	prio 为待删除任务优先级， OS_PRIO_SELF 表示删除当前任务	成功返回 OS_ERR_NONE。不能在中断中调用，也不能删除空闲任务。删除任务前应先释放其持有的互斥量、内存和外设资源。

函数	参数	返回值与作用
<code>OSTaskChangePrio()</code>	<code>oldprio</code> 为原优先级，可用 <code>OS_PRIO_SELF</code> ； <code>newprio</code> 为未占用的新优先级	成功返回 <code>OS_ERR_NONE</code> 。新优先级已占用返回 <code>OS_ERR_PRIO_EXIST</code> ，目标任务不存在返回 <code>OS_ERR_TASK_NOT_EXIST</code> 。改变优先级会立即影响调度顺序。
<code>OSTaskStkChk()</code>	<code>prio</code> 为任务优先级； <code>p_stk_data</code> 指向接收结果的 <code>OS_STK_DATA</code>	成功返回 <code>OS_ERR_NONE</code> ，结果通常包含已用栈和空闲栈。创建任务时必须启用 <code>OS_TASK_OPT_STK_CHK</code> <code>OS_TASK_OPT_STK_CLR</code> ，配置文件也必须启用栈检查功能。

挂起用于临时停止任务，删除用于结束任务，两者不能互相代替。仅为等待条件而挂起任务容易产生“谁负责恢复”的耦合，通常应优先使用信号量或事件标志。

4. 时间管理

```
void OSTimeDly(INT32U ticks);
INT8U OSTimeDlyHMSM(INT8U hours, INT8U minutes,
                    INT8U seconds, INT16U ms);
INT32U OSTimeGet(void);
void OSTimeSet(INT32U ticks);
void OSTimeTick(void);
```

函数	参数	返回值与作用
<code>OSTimeDly()</code>	<code>ticks</code> 为延时节拍数	无返回值。当前任务进入延时态， <code>ticks</code> 为 0 时不延时。不能在中断中调用，调度器锁定时也无法正常阻塞。
<code>OSTimeDlyHMSM()</code>	<code>hours</code> 为小时； <code>minutes</code> 范围 0~59； <code>seconds</code> 范围 0~59； <code>ms</code> 范围 0~999	成功返回 <code>OS_ERR_NONE</code> 。分钟、秒、毫秒越界或总延时为 0 时返回对应错误码；在中断中调用返回 <code>OS_ERR_TIME_DLY_ISR</code> 。最终精度仍受系统节拍限制。
<code>OSTimeGet()</code>	无参数	返回从系统启动或最近一次 <code>OSTimeSet()</code> 以来的 32 位节拍计数。计数会回绕，时间差应使用无符号减法。
<code>OSTimeSet()</code>	<code>ticks</code> 为新的系统节拍计数	无返回值。只修改系统时间基准，不改变已经进入延时态任务的剩余延时。
<code>OSTimeTick()</code>	无参数	无返回值。由周期节拍中断调用，更新系统时间并处理任务延时和超时；普通任务不应主动调用。

`OSTimeDly()` 表达的是，至少等待若干节拍，不是精确定时器。周期任务反复执行“处理工作后再延时固定时间”会累积处理时间，严格周期控制应基于绝对节拍计算下一次唤醒时刻。

5. 信号量

```
OS_EVENT *OSSemCreate(INT16U cnt);
void      OSMemPend(OS_EVENT *pevent, INT32U timeout, INT8U *perr);
INT8U     OSMemPost(OS_EVENT *pevent);
INT16U     OSMemAccept(OS_EVENT *pevent);
```

参数	含义
cnt	信号量初值。事件通知通常设为 0；资源计数设为可用资源数量。最大值由16位计数决定。
pevent	OSMemCreate() 返回的信号量对象指针，不能传 NULL，也不能混用邮箱、队列或互斥量对象。
timeout	等待的最大节拍数；0 表示永久等待。
perr	错误码输出地址，不能传 NULL。

OSMemCreate() 成功时返回信号量指针，事件控制块不足或在中断中创建时返回 NULL。OSMemPend() 无直接返回值，*perr 为 OS_ERR_NONE 表示取得信号量，OS_ERR_TIMEOUT 表示超时，OS_ERR_PEND_ISR 表示在中断中等待，OS_ERR_PEND_LOCKED 表示调度器锁定时等待。

OSMemPost() 发布一次信号量。若有任务等待，则唤醒其中优先级最高者；否则计数加1。成功返回 OS_ERR_NONE，计数已达上限返回 OS_ERR_SEM_OVF。它可在任务或中断中调用。OSMemAccept() 立即尝试获取，返回调用前的计数值；返回 0 表示未取得，不会阻塞。

信号量不记录所有者，不适合代替互斥量保护共享资源。二值事件如果连续多次 Post，计数会累积，接收任务随后可能连续通过多次 Pend。

6. 互斥量

```
OS_EVENT *OSMutexCreate(INT8U prio, INT8U *perr);
void      OSMutexPend(OS_EVENT *pevent, INT32U timeout, INT8U *perr);
INT8U     OSMutexPost(OS_EVENT *pevent);
BOOLEAN   OSMutexAccept(OS_EVENT *pevent, INT8U *perr);
```

参数	含义
prio	优先级上限。该优先级必须高于所有可能访问此互斥量的任务，即数值更小，并且不能被其他任务或互斥量占用。传 OS_PRIO_MUTEX_CEIL_DIS 可关闭优先级提升。
pevent	OSMutexCreate() 返回的互斥量对象指针。
timeout	等待互斥量的最大节拍数；0 表示永久等待。
perr	错误码输出地址。

OSMutexCreate() 成功返回对象指针并令 *perr 为 OS_ERR_NONE；优先级上限已占用、非法或事件控制块不足时返回 NULL。OSMutexPend() 取得互斥量后返回，结果通过 *perr 给出，超时为 OS_ERR_TIMEOUT。

`OSMutexPost()` 只能由当前所有者调用。成功返回 `OS_ERR_NONE`，非所有者释放返回 `OS_ERR_NOT_MUTEX_OWNER`。`OSMutexAccept()` 不阻塞，取得成功返回 `OS_TRUE`，否则返回 `OS_FALSE`，详细结果写入 `*perr`。

互斥量必须遵守“谁取得，谁释放”。不要在持有互斥量期间执行无界等待，也不要在中断中获取或释放互斥量。多个互斥量应规定统一的加锁顺序，以免死锁。

7. 邮箱

```
OS_EVENT *OSMboxCreate(void *pmsg);
void *OSMboxPend(OS_EVENT *pevent, INT32U timeout, INT8U *perr);
INT8U OSMboxPost(OS_EVENT *pevent, void *pmsg);
void *OSMboxAccept(OS_EVENT *pevent);
```

参数	含义
<code>pmsg</code>	消息指针。创建时传 <code>NULL</code> 表示邮箱初始为空；发送时不能传 <code>NULL</code> 。内核只保存指针，不复制指向的数据。
<code>pevent</code>	<code>OSMboxCreate()</code> 返回的邮箱对象指针。
<code>timeout</code>	等待消息的最大节拍数；0 表示永久等待。
<code>perr</code>	错误码输出地址。

`OSMboxCreate()` 成功返回邮箱指针，失败返回 `NULL`。`OSMboxPend()` 成功时返回消息指针并令 `*perr` 为 `OS_ERR_NONE`；超时返回 `NULL` 并置 `OS_ERR_TIMEOUT`。`OSMboxPost()` 成功返回 `OS_ERR_NONE`，邮箱已有未取消息时返回 `OS_ERR_MBOX_FULL`。`OSMboxAccept()` 立即取出消息，邮箱为空时返回 `NULL`。

邮箱只有一个消息位置，不能缓存多条消息。发送局部变量地址、复用同一缓冲区或提前释放动态内存，都会使接收方得到失效或已被改写的数据。

8. 消息队列

```
OS_EVENT *OSQCreate(void **start, INT16U size);
void *OSQPend(OS_EVENT *pevent, INT32U timeout, INT8U *perr);
INT8U OSQPost(OS_EVENT *pevent, void *pmsg);
void *OSQAccept(OS_EVENT *pevent, INT8U *perr);
INT8U OSQFlush(OS_EVENT *pevent);
```

参数	含义
<code>start</code>	指向消息指针数组首元素，例如 <code>void *queue_storage[8]</code> 对应 <code>queue_storage</code> 。该数组只保存指针，必须长期有效。
<code>size</code>	队列可容纳的消息指针数量，不是消息字节数，必须大于0。
<code>pevent</code>	<code>OSQCreate()</code> 返回的队列对象指针。
<code>pmsg</code>	待发送的消息指针，不能为 <code>NULL</code> 。
<code>timeout</code>	等待消息的最大节拍数；0 表示永久等待。

参数	含义
perr	错误码输出地址。

`OSQCreate()` 成功返回队列对象指针，队列控制块或事件控制块不足时返回 `NULL`。`OSQPend()` 成功返回消息指针；超时返回 `NULL` 并置 `OS_ERR_TIMEOUT`。`OSQPost()` 成功返回 `OS_ERR_NONE`，队列已满返回 `OS_ERR_Q_FULL`。`OSQAccept()` 立即取消息，队列为空时返回 `NULL` 并置 `OS_ERR_Q_EMPTY`。`OSQFlush()` 清空全部指针，成功返回 `OS_ERR_NONE`。

队列满时必须明确丢弃、覆盖、报警或流量控制策略。`OSQFlush()` 只清除指针，不释放指针指向的内存；使用动态内存时需先完成资源回收。

9. 事件标志

```
OS_FLAG_GRP *OSFlagCreate(OS_FLAGS flags, INT8U *perr);
OS_FLAGS      OSFlagPend(OS_FLAG_GRP *pgrp, OS_FLAGS flags,
                          INT8U wait_type, INT32U timeout, INT8U *perr);
OS_FLAGS      OSFlagPost(OS_FLAG_GRP *pgrp, OS_FLAGS flags,
                          INT8U opt, INT8U *perr);
OS_FLAGS      OSFlagAccept(OS_FLAG_GRP *pgrp, OS_FLAGS flags,
                           INT8U wait_type, INT8U *perr);
```

参数	含义
pgrp	<code>OSFlagCreate()</code> 返回的事件标志组指针。
flags	创建时表示各标志位初值；等待时表示关注的位掩码；发布时表示需要置位或清零的位。
wait_type	等待条件。 <code>OS_FLAG_WAIT_SET_ALL</code> 要求指定位置位， <code>SET_ANY</code> 要求任一位置位， <code>CLR_ALL</code> 要求全部清零， <code>CLR_ANY</code> 要求任一位置清零。可按位或 <code>OS_FLAG_CONSUME</code> ，使满足条件的标志在成功取得后自动反向清除或置位。
timeout	等待条件的最大节拍数；0 表示永久等待。
opt	<code>OS_FLAG_SET</code> 把指定位置1， <code>OS_FLAG_CLR</code> 把指定位置清0。
perr	错误码输出地址。

`OSFlagCreate()` 成功返回标志组指针，资源不足时返回 `NULL` 并置 `OS_ERR_FLAG_GRP_DEPLETED`。`OSFlagPend()` 返回满足条件的标志位，成功时 `*perr` 为 `OS_ERR_NONE`，超时为 `OS_ERR_TIMEOUT`，等待方式错误为 `OS_ERR_FLAG_WAIT_TYPE`。`OSFlagPost()` 返回操作后的标志值。`OSFlagAccept()` 与 `Pend` 判断规则相同，但不阻塞；条件不满足时置 `OS_ERR_FLAG_NOT_RDY`。

事件标志组只能表达条件位，不能携带消息内容。使用 `OS_FLAG_CONSUME` 前应确认事件是否允许被一个任务消费；多个任务都需要观察同一状态时，自动清除可能使后续任务看不到该状态。

###

- `Create` 和 `Del` 类函数通常只能在任务环境调用；`Pend` 类函数只能在任务环境调用，并且调用时调度器不能处于锁定状态。
- `Post` 类函数多半允许在中断中调用，但互斥量接口除外。中断中发布事件后，实际任务切换由 `OSIntExit()` 统一处理。

- 返回对象指针的创建函数应同时检查返回值和错误码；返回 void 的 Pend 函数必须检查 *perr。
- 多数接口受 os_cfg.h 中的 OS_XXX_EN 控制。配置为 0 时，相应代码不会参与编译，调用该函数会出现声明或链接错误。
- 对象删除后，原指针立即失效。等待任务可能以 OS_ERR_PEND_ABORT 返回，不能继续使用原对象。
- uC/OS-II 传递的邮箱和队列消息都是指针，缓冲区所有权和生命周期必须由应用程序明确约定。

九、移植

uC/OS-II 内核主体与 CPU 关系较弱，但任务切换、栈初始化、中断开关和异常处理依赖 CPU 架构和编译器。

移植层通常包括：

内容	作用
OS_STK 类型	定义任务栈元素类型
栈增长方向	决定栈顶和栈底的使用方式
临界区宏	控制中断开关或保存中断状态
任务栈初始化	构造任务首次运行需要的寄存器现场
上下文切换	保存当前任务现场并恢复下一个任务现场
节拍接口	把定时中断接入内核时间管理

注意事项：

- 不同 CPU 或不同编译器的移植文件不能随意混用；
- 汇编文件与 C 文件中的函数名、调用约定必须一致；
- 系统节拍中断必须稳定，否则延时和超时都不可靠；
- 能调用内核接口的中断优先级必须符合移植层要求。

十、常见问题

问题	原因
任务创建失败	优先级重复、栈无效、任务数量超过配置、功能未启用
任务长期不运行	优先级过低、高优先级任务不阻塞、任务被挂起
系统时间不增加	节拍中断未进入、节拍接口未调用、中断配置错误
延时不准确	节拍频率设置不当、任务处理时间过长、中断被长时间屏蔽
共享变量偶发错误	缺少临界区、缺少互斥保护、任务和中断同时访问
栈溢出	栈太小、局部对象过大、函数调用过深、浮点现场未考虑
优先级反转	低优先级任务持有资源，高优先级任务等待，中优先级任务抢占
死锁	多个任务以不同顺序等待多个资源